

Bonus – Rappel Compilation OCaml

En OCaml, un fichier définit un module, dont le nom est celui du fichier avec une majuscule comme première lettre (le reste de la casse est respectée).

Rappel .1. Compilation OCaml

Pour compiler un fichier en OCaml, on utilise `ocamlc`. En particulier, pour plusieurs fichiers `main.ml`, `util1.ml` et `util2.ml` en un exécutable `nom_executable`, tel que `main.ml` a besoin de `util2.ml` qui a lui-même besoin de `util1.ml` on peut utiliser la commande:

```
> ocamlc util1.ml util2.ml main.ml -o nom_executable
```

Il est aussi possible de faire une compilation séparée via l'option `-c`, on obtient alors des fichiers `.cmo` et `.cmi`:

```
> ocamlc -c util1.ml
> ocamlc -c util1.cmo util2.ml
> ocamlc util1.cmo util2.cmo main.ml -o nom_executable
```

Pour utiliser la fonction `f`, définie dans `util1.ml`, dans le fichier `main.ml` il faut appeler `Util1.f`.

Les fichiers `.cmi` sont des fichiers d'interface. Ils permettent entre autre à votre éditeur, via Merlin notamment, de vous proposer l'autocomplétion pour les modules ainsi définis. Merlin est généralement installé par défaut par l'OCaml LSP (Language Server Protocol). Sous VS Code, il peut être nécessaire, une fois le LSP installé, de lui préciser d'utiliser un fichier `.merlin`: OCaml > Server > Args et lui donner dans le JSON ainsi ouvert:

```
"ocaml.server.args": ["--fallback-read-dot-merlin"]
```

Pour les TP, le `.merlin` peut juste contenir:

```
S .
B .
```